L17: Practice of Backpropagation

C. V. Jawahar

IIIT Hyderabad

March 23, 2026

Backpropagation:Recap

- **Step 0:** Initialize the Network (MLP), weights, W^0.
- **Step 1:** Do forward pass for a mini batch (B) of randomly selected samples. Predict outputs with the existing weights.

$$o_i = f(\mathbf{W}, x_i)$$

- **Step 2:** Compute Loss for the set of samples.

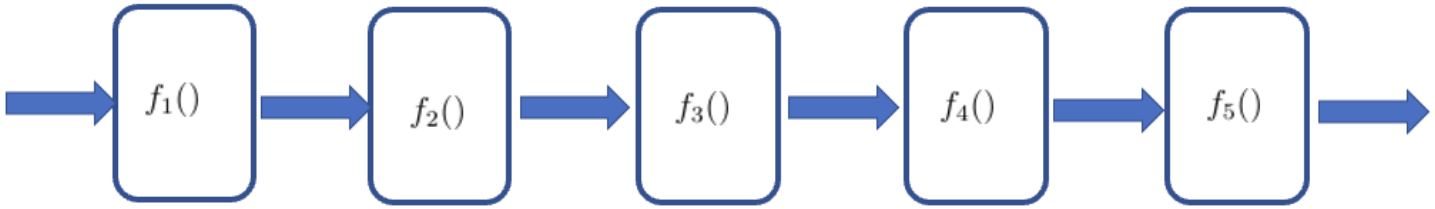
$$L = \frac{1}{|B|} \sum_{i \in B} \mathcal{L}(o_i, y_i)$$

- **Step 3:** Update all the weights using gradient descent.

$$W^{k+1} \leftarrow W^k - \eta \frac{\partial L}{\partial W^k}$$

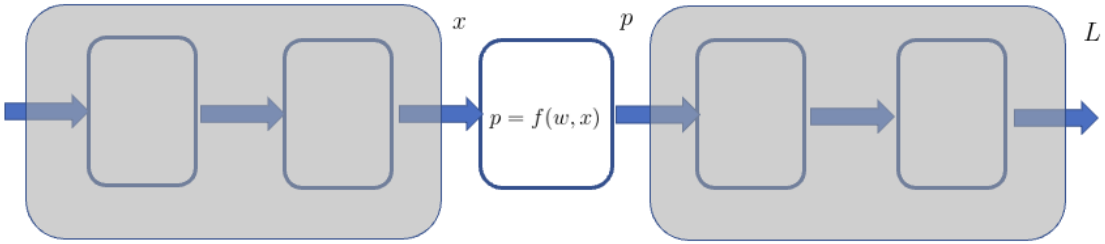
- **Step 4:** Repeat all steps (until the loss is acceptable.)

Notes: Recap



- Two major steps (i) Forward Pass (ii) Backward Pass.
- If we compute gradients backward, it is efficient. "We propagate gradients backwards."
- Some blocks in the chain/graph may have learnable parameter and some may not have.
- Softmax, Activations, Loss etc. can be seen as layers (or some computational functions/blocks in the network).
- Long chain of multiplications can lead to "Vanishing gradients" or "exploding gradients".

Notes: Recap



- For every layer we use, we need to know how to compute derivative of output wrt input (i.e., $\frac{\partial p}{\partial x}$) so that gradients can be propagated backward.

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial x}$$

- For every layer, we need to know how to compute derivative of output wrt parameters/weights (i.e., $\frac{\partial p}{\partial w}$), (if we have) so that weights can be updated or we can train/learn.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial w}$$

Problem

Plot derivative of Sigmoid. How does ReLU help in vanishing gradients?

Opportunities over Simple BP

Backpropagation algorithm has been refined significantly with many minor but critical innovations, for efficiency and practicality.

What all can change in the simple version of BP we had seen?

1. **option 1:** Initialization can be smarter. Initialization will matter, specially given that this is non-convex.
2. **option 2:** Computing loss over a full batch and updating it once is not the best. Why do we need loss? (or an estimate of the loss is sufficient?)
3. **option 3:** Loss function can be different. There re many other loss functions available beyond MSE. We can add more desirability so that it less overfits.
4. **option 4:** Update rule can be different. What can change in gradient desent equation?

Opportunities over Simple BP

- Better Initialization
- Effective Gradient Estimates
- Better Update Rule: Momentum
- Choice of Optimizer
- Loss Function
- Termination
- More ?

Initialization

- **Random** Random breaks symmetry issues
- **Xavior** Xavier initialization is designed to keep the variance of activations and gradients roughly the same across layers. It prevents gradients from vanishing or exploding by setting weights to random values with a mean of 0 and variance related to the fan-in. It is good for optimizing training for sigmoid or tanh activations
- **He** Weights are initialized with a normal distribution having mean 0 and variance related fan-in/fan-out. .

Q: Find out exact equations and demonstrate the utility of initialization. (more in next lecture)

Problem

1. Is it a good idea to initialize a NN with all weights as zero or equal? Why?

Effective and Efficient Gradient Estimates

- **Batch**
- **Stochastic Gradient Descent (Single Sample)**
- **Stochastic Mini Batch GD**

Epoch and Iterations

- **Epoch:** One complete sweep through the entire dataset.
- **Iteration:** One batch processed and one weight update.
- **Batch Size:** The number of samples processed in one iteration.

$$\text{Iterations per Epoch} = \frac{\text{Total Training Samples}}{\text{Batch Size}}$$

Learning Rate

$$W^{k+1} \leftarrow W^k - \eta \frac{\partial L}{\partial W^k}$$

- What will be a good learning rate? Higher? lower?
- **Learning Rate Schedulers**dynamically adjusts the learning rate during training to improve model convergence and performance, usually by reducing it over time
  - **Step Decay:** Reduces the learning rate by a constant factor every few epochs
  - **Exponential Decay:** Decreases the learning rate exponentially based on the epoch number.
  - **Cosine Annealing:** Reduces the learning rate following a cosine curve, often used to escape local minima.
  - **Reduce on Plateau:** Lowers the learning rate when a monitored metric (e.g., validation loss) stops improving.
  - **One Cycle/Cyclic Learning Rate:** Cycles the learning rate between two bounds, often increasing then decreasing, to accelerate

Momemntum

Momentum

$$\theta^{k+1} \leftarrow \theta^k - \Delta\theta$$

$$\Delta\theta = \eta \frac{\partial L}{\partial \theta}$$

$$\Delta\theta = \eta \frac{\partial L}{\partial \theta} + \gamma \Delta\theta^{t-1}$$

Regularization

- Enhance Loss functions
  - $\mathcal{L}' = \mathcal{L} + ||\mathbf{w}||$
  - L1/L2 regularization
- Data Augmentation: Enhance Data:ModelParameter Ratio
- Add noise: Robustify. Where? Input, Output, Weights.
- Drop out (TBD)
- Normalization (TBD)

(Batch) Normalization

- We normalize the input data before feeding to the NNs
- What about the input distribution at each hidden layer? the distribution changes with iterations internal co-variate shift
- Solution: Normalize the layer's input over a mini-batch

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

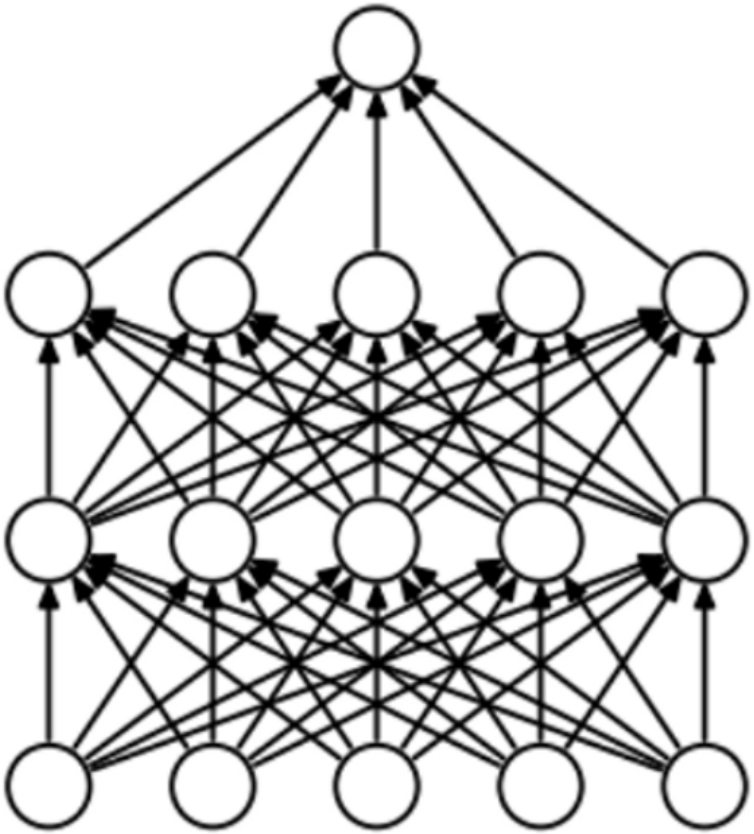
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \qquad \text{// normalize}$$

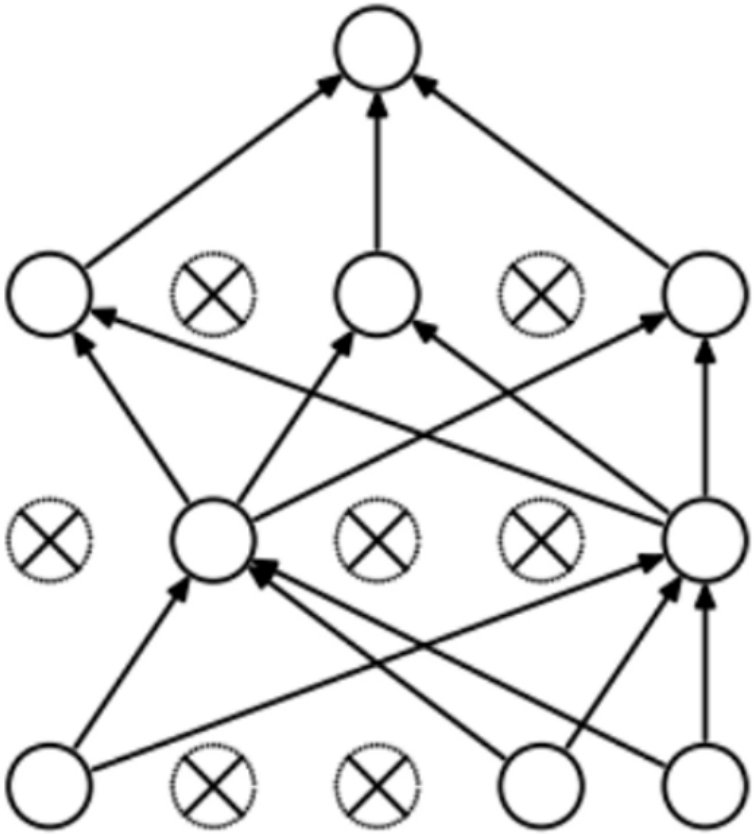
$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

DropOut



(a) Standard Neural Net



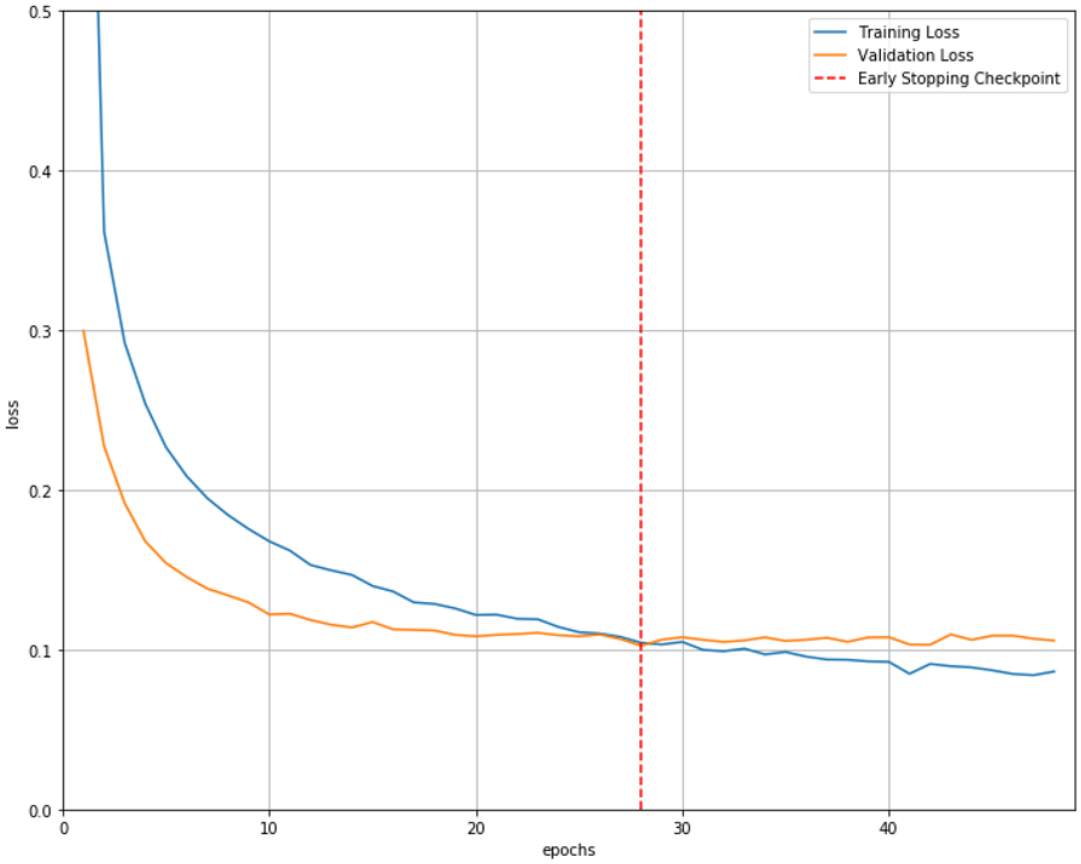
(b) After applying dropout.

DropOut

- Ignore/delete/mask certain neurons while training.
  - Get a simpler network. Eg. Multiply the outputs by 0 or 1 at random.
  - Equivalent to creating many neural networks.
- Reduces complex co-adaptations of neurons, since a neuron cannot rely on the presence of particular other neurons.
- Force to learn more robust features that are useful in conjunction with many different random subsets of the other neurons.
- Set the output of each hidden neuron to zero with a probability of p (say 0.5).
- The neurons which are "dropped out" in this way do not contribute to the forward pass and do not participate in back propagation.
- Thus neural network samples a different architecture, but all these architectures share weights.
- At test time, scale outputs by probability p.

Early Stop

- Stop the training if validation loss/accuracy not improving
- Avoids overfitting on the training data



Problem

What do we mean by overfitting? If we train a network for long time, will we always reach zero loss?